点亮一个LED / LED闪烁 / LED流水灯

对应江协科技《51单片机入门教程-2020版》P4《[2-1]点亮一个LED》、P5《[2-2]LED闪烁》、P6《[2-3]LED流水灯》· 本章目标：会建Keil工程点亮LED、理解共阳低电平点亮、用软件延时做闪烁与流水灯 · 16 题分类：一、概念填空（1-5）；二、判断改错（6-9）；三、计算（10-12）；四、编程应用（13-16）

一、概念填空（1–5）

1. 直插式LED判别正负极：未剪引脚时，长脚是_____极，短脚是_____极；看内部金属片，小块（小红旗）是_____极，大块（大红旗）是_____极。

答：正；负；正；负。

思路：方法/口诀："长正短负、小正大负"。**步骤：**① 引脚没剪时直接比长短；② 剪过脚看内部金属片，小块为阳极(正)、大块为阴极(负)；③ 只有正接正、负接负才有电流流过，接反不亮。

常见错法：把内部大块当成正极；或记反长脚短脚。

解析：LED本质是二极管，单向导通，只有阳极电位高于阴极时才有电流。开发板用贴片LED，背面箭头也标示方向；接反整个灯不亮，是初学者排查"灯不亮"时的第一检查点。

2. 开发板LED接法：8个LED的正极经限流电阻接到_____，负极接到单片机的_____口；由此推得：引脚输出_____电平时LED点亮，输出_____电平时熄灭。

答：VCC(+5V)；P2；低(0V)；高(5V)。

思路：方法/口诀："共阳接VCC，低电平点亮"。**步骤：**① 正极已经电阻接到+5V；② 当引脚拉到0V，电流从VCC经电阻、LED流入引脚，灯亮；③ 引脚拉到5V，LED两端等电位、无电流，灯灭。

常见错法：以为跟"LED负极接地、正极接IO"的接法一样高电平点亮，写 P2=0xFF 想点亮8个灯，结果全灭。

解析：本板是共阳接法。8个LED全接在P2口，P2.0对应D1；要哪个灯亮，就把那一位写成0。

3. 51单片机的电平规范叫_____电平，其中高电平=_____V，低电平=_____V；头文件把P2寄存器取名对应地址_____（十六进制）。

答：TTL；5；0；0xA0。

思路：方法/口诀："TTL高5低0"；"P2地址0xA0"。**步骤：**① 51口线只认两种状态：5V与0V；② 头文件 reg52.h 把各寄存器地址取好名字，P2对应0xA0；③ 包含头文件后编译器才认识P2。

常见错法：把高电平记成3.3V（那是另一些平台的标准）；或不知道P2对应0xA0。

解析：51的IO口是数字量，非5V即0V，没有中间值。CPU往P2这个地址写8位数据，每一位经驱动器连一根IO线，写1出5V、写0出0V。

4. 编译后报"P2是未定义的标识符"，是因为main之前没有_____；编译器默认不生成下载文件，必须在Options→Output勾选_____，STC-ISP才能烧录；主函数跑完单片机会_____，所以main末尾要写_____把程序兜住。

答：包含 reg52.h 头文件；Create HEX File；从头重新执行；while(1) 死循环。

思路：方法/口诀："缺头文件、勾HEX、加死循环"三件套。**步骤：**① #include <reg52.h> 让P2有定义；② 勾 Create HEX File 才会产出 .hex；③ while(1) 非0即真、永远成立，程序停在里面兜圈不重跑。

常见错法：把"P2未定义"当成软件故障去重装Keil；忘了勾HEX到处找 .hex；写完main不写 while(1) 导致赋值被反复重跑。

解析：这是P4最容易踩的三个工程坑。注意每次新建工程都要重新勾一次 Create HEX File，这个勾选不随工程复制。

5. 排阻上标字"102"，读法是前两位有效数字、第三位当倍率（后面加几个0），则102=_____Ω=_____kΩ；C语言不能直接写二进制字面量，要把二进制按每_____位转成一位十六进制来写。

答：1000；1；4。

思路：方法/口诀："两位有效一位倍，四位二进制一位十六进制"。**步骤：**① $102 = 10 \times 10^2 = 1000\Omega$；② 限流电阻串在VCC与LED之间，防止5V直接加在LED上烧管；③ 4位二进制(0000~1111)正好对应一位十六进制(0~F)。

常见错法：把102读成102Ω；或不理解为什么要串限流电阻。

解析：限流电阻1kΩ是本板固定设计。十六进制是二进制的简写，P2=0xFE 里的 FE 就是 1111 1110 的紧凑写法，避免写一长串0和1。

二、判断改错（6–9）

6. 判断：在C里写 `P2=11111110;` 就能直接点亮D1。

答：错误。改正：C语言不支持二进制字面量，11111110会被当成十进制数11111110处理；应把二进制转成十六进制，写 `P2=0xFE;`（=1111 1110）。

思路：方法/口诀："二进制不直写，四位并一位转十六进制"。**步骤：**① C的字面量只有十进制、0x开头十六进制、0开头八进制，没有二进制；② 1111 1110 从右每4位分组：1111=F，1110=E，得0xFE；③ 改写 P2=0xFE。

常见错法：以为C跟某些语言一样有0b前缀；写出11111110后数值完全不对。

解析：这就是为什么要学十六进制——51寄存器都是8位，用二进制看亮灭最直观，但写程序时必须落到十六进制字面量，如 0xFE、0xFF。

7. 判断：编译报"P2未定义的标识符"说明Keil装坏了，重装软件即可。

答：错误。改正：这是因为没有包含头文件；在main之前写 `#include <reg52.h>`，头文件里已把P0/P2/SP等寄存器地址（P2对应0xA0）取好名字，包含后即0 error。

思路：方法/口诀："报未定义先查头文件"。**步骤：**① 看报错是不是标识符未定义；② 检查有没有 #include <reg52.h>；③ 补上重新Build。

常见错法：重装Keil浪费时间；或自己用别的关键字去"定义P2"。

解析：头文件相当于一张寄存器名字表，reg52.h把0xA0这个寄存器起名叫P2，编译器才认识。另外Keil器件列表没有STC，要选 Atmel→AT89C52，二者操作和功能一样。

8. 判断：用STC-ISP软件延时计算器时，指令集保持默认Y5就行。

答：错误。改正：本板是89系列（Keil选AT89C52、STC-ISP选STC89C52），指令集必须选Y1（最老指令集）；默认Y5适用于STC15系列，不匹配会导致延时时间不准。系统频率也要填本板晶振 12.000MHz。

思路：方法/口诀："89系列选Y1、频率填12M"。**步骤：**① 打开软件延时计算器；② 选指令集Y1、频率12.000MHz、所需延时；③ 生成代码后粘贴进工程。

常见错法：用默认Y5导致延时差好几倍，还以为是程序逻辑错。

解析：不同内核指令执行周期不同，计算器选对指令集和晶振频率，才能生成时间准确的空转函数。

9. 判断：51单片机里int和电脑上一样是32位。

答：错误。改正：51单片机里int是16位（2字节），unsigned int范围0~65535，默认signed int范围-32768~32767；电脑上的int才是32位。char是8位=1字节，unsigned char范围0~255。

思路：方法/口诀："51的int是16位，别拿电脑经验套"。**步骤：**① 记2^{16}=65536个数；② 无符号从0数到65535；③ 带符号对半分 -32768~32767。

常见错法：把int当32位算范围；或以为unsigned char能存256。

解析：变量是不同大小的"数据盒子"，51资源紧，int只有16位。带参延时函数形参用 unsigned int x，正好能装0~65535，够用。

三、计算（10–12）

10. 进制互转：把下列十六进制写成8位二进制；再把 1111 1101 写成十六进制。(1) 0xFE　(2) 0x55　(3) 0x7F。

答：0xFE = 1111 1110；0x55 = 0101 0101；0x7F = 0111 1111；1111 1101 = 0xFD。

思路：方法/口诀："一位十六进制展开成四位二进制，0~9照写、A=10…F=15"。**步骤：**① F=1111、E=1110，拼1111 1110；② 5=0101，两个5得 0101 0101；③ 7=0111、F=1111，得 0111 1111；④ 反着分组 1111=F、1101=D，得 0xFD。

常见错法：展开时不足4位补0补错；或把E当成别的值。

解析：P2写0xFE只有最低位是0，所以只亮D1；写0x55是隔一位亮一个，可用来做间隔亮灯测试。

11. 读电阻：写出下列排阻标字对应的阻值。(1) 102　(2) 473　(3) 1001。

答：102 = 10×10^2 = 1000Ω = 1kΩ；473 = 47×10^3 = 47000Ω = 47kΩ；1001（四位标注）= 前三位有效100 × 倍率10^1 = 1000Ω = 1kΩ。

思路：方法/口诀："三位标：前两位有效、第三位倍率；四位标：前三位有效、第四位倍率"。**步骤：**① 102：10后面补2个0；② 473：47后面补3个0；③ 1001是四位标注，100后面补1个0。

常见错法：把1001仍按三位读；或补0个数数错。

解析：本板LED限流电阻就是102=1kΩ，串在+5V与LED之间限流，防止电流过大烧毁LED。

12. 流水灯序列与数据范围：已知第1个LED亮时 P2=0xFE（那个0在最低位），把这个0逐位左移，依次写出8个灯各亮时P2应赋的十六进制；再算：51单片机 unsigned int 的最大值 $2^{16}-1$ = ？ unsigned char 的最大值 2^8-1 = ？

答：左移序列 0xFE→0xFD→0xFB→0xF7→0xEF→0xDF→0xBF→0x7F；unsigned int 最大值 = 65535；unsigned char 最大值 = 255。

思路：方法/口诀："亮位0逐位左移：FE FD FB F7 EF DF BF 7F"；"n位无符号最大值=2^n-1"。**步骤：**① 0从bit0移到bit1…bit7；② 每移一位对应一位十六进制变化，得8个值；③ 2^{16}=65536、减1得65535；2^8=256、减1得255。

常见错法：序列中间跳错（如把0xF7后面写成0xF8）；或把$2^{16}-1$算成65536。

解析：左移一位二进制就是 1111 1110 变 1111 1101（0xFE→0xFD），肉眼看灯从D1向D8方向流动。带参延时形参 unsigned int x 最大65535ms，足够用。

四、编程应用（13–16）

13. 读程找错：下面这段自写程序想点亮D1，编译与运行都有问题，请找出两处错误并改正。

```
void main(void)
{
    P2 = 0xFE;
}
```

答：两处错误——① 缺 `#include <reg52.h>`，P2未定义，编译报错；② main末尾没有 while(1) 死循环，主函数跑完单片机会从头重新执行。改正：

```
#include <reg52.h>
void main(void)
```

```
{
    P2 = 0xFE;
    while(1){}
}
```

思路：方法/口诀："工程三件套：头文件、赋值、死循环"。**步骤：**① 检查是否包含 reg52.h；② 检查main里是否给P2赋了正确值0xFE；③ 末尾是否 while(1) 兜住。

常见错法：只看赋值没写够，漏掉头文件；或写了 while(1) 却把 P2=0xFE 的位置放错。

解析：哪怕这里只赋值一次，没有 while(1) 单片机也会反复从头跑，属不规范写法；加上后程序停在死循环里，D1保持亮。

14. 编程：只点亮D1，其余7个LED熄灭。写出完整Keil C51程序。

答：

```
#include <reg52.h>   /* 头文件，定义P2等寄存器 */

void main(void)
{
    P2 = 0xFE;        /* 1111 1110：最低位P2.0=0，D1亮，其余位=1熄灭 */
    while(1){}        /* 死循环，停住不重跑 */
}
```

思路：方法/口诀："共阳接法：亮的那一位写0，其余写1"。**步骤：**① D1接P2.0、低电平点亮；② 8位里只有P2.0为0：1111 1110 = 0xFE；③ 写 while(1)。

常见错法：写成 P2=0x01 想"点亮第1位"，结果只有D1灭、其余全亮。

解析：这就是[2-1]的核心效果。下载前记得Options→Output勾 Create HEX File；STC-ISP里选STC89C52、打开.hex、下载后冷启动（断电再上电）才烧进去。代码为自写示例（非原课程源码）。

15. 编程：让D1以1秒为周期闪烁（亮500ms、灭500ms）。写出完整程序，延时函数由STC-ISP计算器生成。

答：

```
#include <reg52.h>
#include <intrins.h>   /* 延时代码若用到 _nop_() 则需要 */

/* Delay500ms() 由 STC-ISP 软件延时计算器生成：
   系统频率 12.000MHz、指令集 Y1、延时 500ms；
   函数体为 do-while 空转计数，此处不抄写计算器原始代码。 */
void Delay500ms(void)
{
    /* 计算器生成的约 500ms 空转循环 */
}
```

```
void main(void)
{
    while(1)
    {
        P2 = 0xFE;    /* D1 亮 */
        Delay500ms(); /* 亮保持 500ms */
        P2 = 0xFF;    /* 全灭（D1 也灭） */
        Delay500ms(); /* 灭保持 500ms */
    }
}
```

思路：方法/口诀："亮一句、延500，灭一句、延500，合起来1秒一周期"。**步骤：**① P2=0xFE 亮D1；② 调Delay500ms()；③ P2=0xFF 全灭；④ 再调 Delay500ms()；⑤ 整体放进 while(1)。

常见错法：不写延时直接 P2=0xFE / P2=0xFF 来回切，结果LED一直亮只是不太亮（MCU是MHz级、每秒约百万次，肉眼分辨不出）；或延时计算器指令集选成Y5、频率没填12MHz导致时间不准。

解析：单片机跑在兆赫兹量级，亮灭两句之间不延时，切换太快肉眼看上去就是一直亮。亮500ms+灭500ms正好一个完整周期1秒。代码为自写示例（非原课程源码），延时空转体以STC-ISP生成结果为准。

16. 编程：让8个LED依次向左流水（D1→D8），并用带参数的通用延时函数控制节奏（每灯停留500ms，想变速只需改传参）。写出完整程序。

答：

```
#include <reg52.h>
#include <intrins.h>

/* Delay1ms(unsigned int x)：先由 STC-ISP 计算器生成约 1ms 空转段
   （系统频率 12.000MHz、指令集 Y1），再套一层 while(x) 实现 x 毫秒延时。 */
void Delay1ms(unsigned int x)   /* x 范围 0~65535 */
{
    while(x)
    {
        /* 计算器生成的约 1ms 空转段 */
        x--;
    }
}

void main(void)
{
    while(1)
    {
        P2 = 0xFE; Delay1ms(500);  /* D1 亮 */
        P2 = 0xFD; Delay1ms(500);  /* D2 亮 */
        P2 = 0xFB; Delay1ms(500);  /* D3 亮 */
        P2 = 0xF7; Delay1ms(500);  /* D4 亮 */
```

```
        P2 = 0xEF; Delay1ms(500);  /* D5 亮 */
        P2 = 0xDF; Delay1ms(500);  /* D6 亮 */
        P2 = 0xBF; Delay1ms(500);  /* D7 亮 */
        P2 = 0x7F; Delay1ms(500);  /* D8 亮 */
    }
}
```

思路：方法/口诀："亮位0左移，FE FD FB F7 EF DF BF 7F；带参延时传几延几毫秒"。**步骤：**① 先做1ms延时函数再加形参x，while(x){ 1ms空转; x--; }；② 依次写8个P2赋值，每个后面 Delay1ms(500)；③ 想变快就把500改成100、变慢改成1000，甚至不同灯传不同值做变速。

常见错法：把带参延时形参写错类型；或序列写错导致灯乱跳。

解析：复制粘贴8行是最朴素的流水灯写法，直观但花样多就繁琐；改成带参延时后，调速只改 Delay1ms 里的数字，不用重新生成延时函数。代码为自写示例（非原课程源码），1ms空转体以STC-ISP生成结果为准。